

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO BÀI TẬP LỚN
LẬP TRÌNH MẠNG

**Đề tài: Ironfront Reborn - Trò chơi bắn súng nhiều người chơi
với tầng mạng TCP/UDP tự xây dựng**

Lớp: D23CTPM01-B
Nhóm bài tập lớn: 10
Sinh viên thực hiện: Nguyễn Tự Kiên
Mã sinh viên: B23DCCN465

Báo cáo kiến trúc và giao diện hệ thống
Phần cá nhân: *Phân hệ Master Server TCP và hạ tầng*

Hà Nội - 2026

Mục lục

I. NỘI DUNG NHÓM	3
1. Mô tả kiến trúc chung của hệ thống	3
1.1. Bài toán và phạm vi	3
1.2. Kiến trúc ba tiến trình	3
1.3. Phân chia TCP và UDP theo đặc tính dữ liệu	5
1.4. Kiến trúc phân lớp hệ thống	5
1.5. Kiến trúc triển khai	6
1.6. Phân công theo phân hệ	7
2. Mô tả thiết kế chung của hệ thống	7
2.1. Mô hình thẩm quyền	7
2.2. Luồng nghiệp vụ đầu - cuối	8
2.3. Các kỹ thuật netcode cốt lõi	9
2.4. Thiết kế giao thức	9
2.5. Tham số thời gian	10
2.6. Phân loại đối tượng đồng bộ	11
2.7. Bảo mật ở mức phù hợp phạm vi	11
2.8. Các quyết định kiến trúc đã chốt	11
2.9. Quy trình phát triển và bảo đảm chất lượng	11
II. NỘI DUNG CÁ NHÂN - Phân hệ Master Server TCP và hạ tầng.	13
3. Biểu đồ và giao diện nội dung cá nhân thực hiện	13
3.1. Kiến trúc thành phần Master Server	13
3.2. Đóng khung tin trên luồng TCP	13
3.3. Luồng dịch vụ sảnh chờ	15
3.4. Vòng đời kết nối và phòng	16
3.5. Mô hình dữ liệu	16
3.6. Giao diện lập trình và giao diện giám sát	17
3.7. Hạ tầng triển khai	18
4. Luồng xử lý và giải pháp đã ứng dụng	18
4.1. Luồng tiếp nhận kết nối và đọc khung tin	18
4.2. Luồng đăng ký và đăng nhập	21
4.3. Luồng danh sách phòng, tạo phòng, vào phòng và cấp vé	23
4.4. Luồng ghép trận nhanh	24
4.5. Luồng chat trong sảnh chờ	25
4.6. Luồng game server: đăng ký, báo sống, báo kết quả trận và sự cố	26
4.7. Luồng ngắt kết nối và dọn dẹp	27
4.8. Bảo mật kênh TCP bằng TLS	27
4.9. Tổng hợp kiểm thử và số đo	28
4.10. Bảng ảnh xạ hình vẽ với chức năng cá nhân thực hiện	28

I. NỘI DUNG NHÓM

1. Mô tả kiến trúc chung của hệ thống

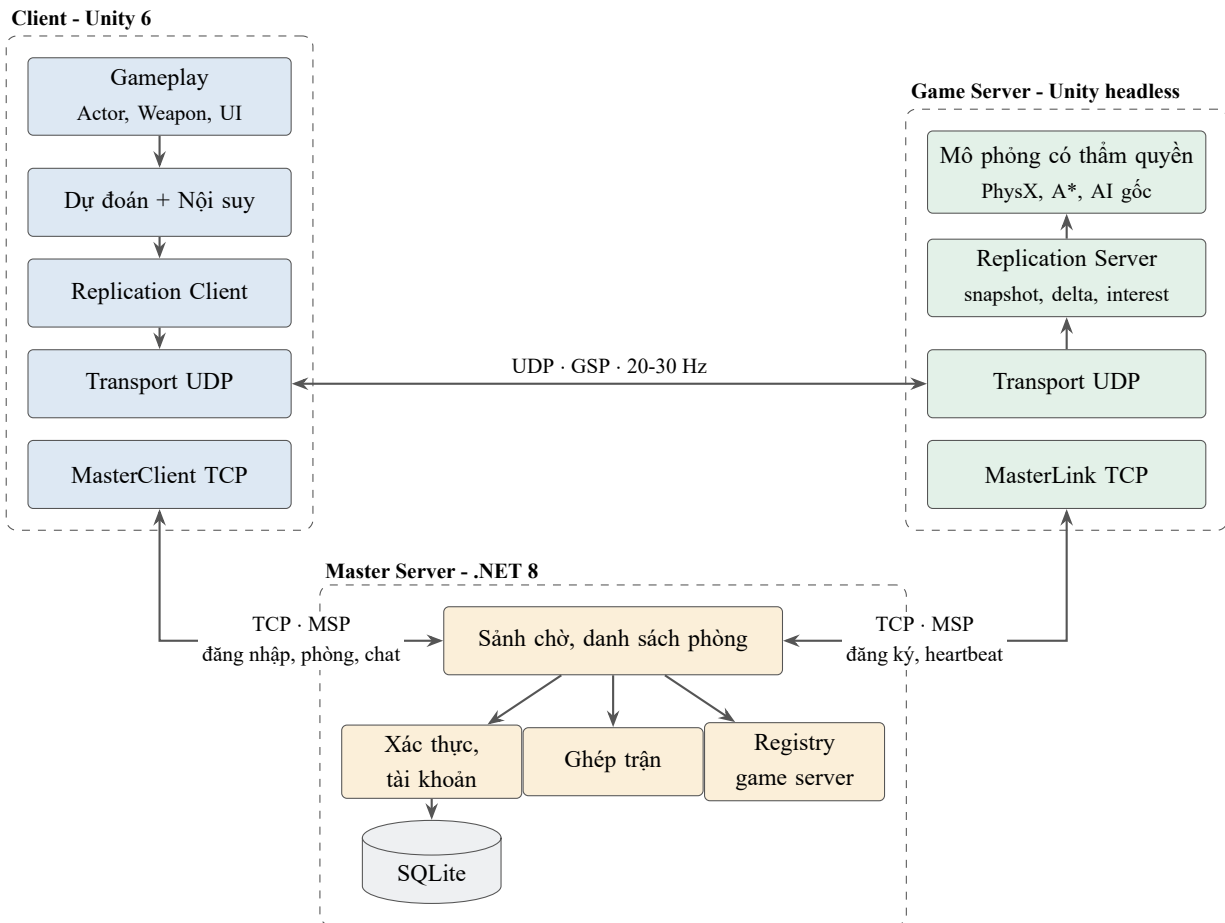
1.1. Bài toán và phạm vi

Ironfront Reborn chuyển mã nguồn trò chơi bắn súng góc nhìn thứ nhất *Ravenfield Beta 5* (Unity 6000.3, vốn chỉ chơi đơn) thành một trò chơi **hiều người chơi, máy chủ nắm quyền quyết định** (server-authoritative). Yêu cầu cốt lõi của môn Lập trình mạng: **toàn bộ tầng mạng được viết tay trên socket TCP và UDP thô** của hệ điều hành - không dùng Mirror, Photon, Netcode for GameObjects, KCP, gRPC hay WebSocket.

Tiêu chí	Giá trị thiết kế
Quy mô một trận	16 người chơi thật + 32 bot AI (tối đa 64 actor, 16 phương tiện)
Tần số mô phỏng / snapshot / gửi input	30 Hz / 20 Hz / 30 Hz
Băng thông xuống mỗi client	≤ 8 KB/s (đo trên server đã triển khai: 1,67 KB/s)
Triển khai	LAN trước, sau đó VPS công cộng (Docker Compose, TLS cho kênh TCP)
Công nghệ	.NET 8 (master server, công cụ), netstandard2.1 (thư viện dùng chung), Unity 6 (client và game server headless)

1.2. Kiến trúc ba tiến trình

Hệ thống gồm **ba loại tiến trình** độc lập, giao tiếp qua **hai giao thức tự định nghĩa**: **MSP** (Master Server Protocol, trên TCP) cho dịch vụ săn chờ và **GSP** (Game Server Protocol, trên UDP) cho lưu lượng trong trận. Hai giao thức gặp nhau ở đúng một điểm: **joinTicket** được master ký HMAC và game server tự kiểm tra.



Hình 1.1. Kiến trúc tổng thể ba tiến trình và hai giao thức MSP/GSP

Tiến trình	Nền tảng	Trách nhiệm chính	Giao tiếp
Client	Unity 6, bản build thường	Hiển thị, nhận input, dự đoán chuyển động người chơi cục bộ, nội suy actor ở xa, giao diện sảnh và HUD	TCP tới Master; UDP tới Game Server
Game Server	Unity 6 headless (chế độ không đồ họa)	Chạy mô phỏng có thẩm quyền (vật lý, AI, bắn, điểm chiếm), tạo snapshot riêng cho từng client, bù trễ	Nhận UDP từ client; TCP tới Master
Master Server	.NET 8 console, SQLite	Tài khoản, phiên, phòng, ghép trận, chat sảnh, registry game server, cấp joinTicket	Nhận TCP từ client và game server

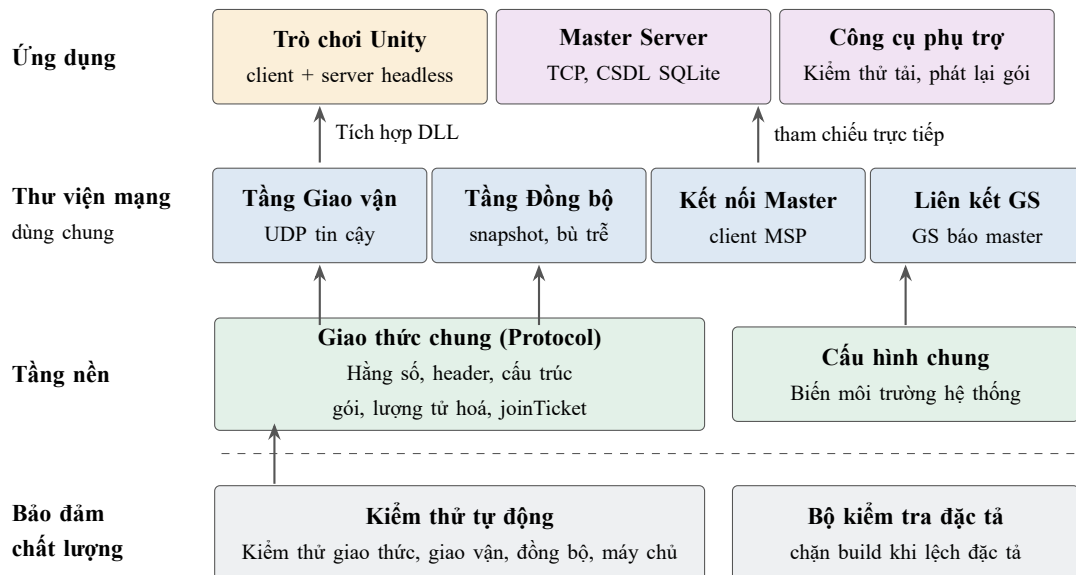
1.3. Phân chia TCP và UDP theo đặc tính dữ liệu

Đặc tính	TCP - Master Server	UDP - Game Server
Dùng cho	Đăng nhập, danh sách phòng, tạo/vào phòng, ghép trận, chat sảnh, heartbeat server	Mọi lưu lượng trong trận: input, snapshot, sự kiện gameplay
Tần suất	Vài gói/phút mỗi client	20-60 gói/giây mỗi chiều
Mất gói	Không chấp nhận (mất LOGIN_RES là treo màn hình)	Chấp nhận với trạng thái: snapshot sau thay thế snapshot trước
Nếu đổi chỗ	Phải tự viết lại độ tin cậy mà TCP đã làm tốt	Head-of-line blocking: một gói mất chặn mọi gói sau; Nagle + delayed ACK thêm 40-200 ms
Đóng khung	Tiền tố độ dài 4 byte + JSON	Datagram tự nhiên, header nhị phân 16 byte

Không dùng WebSocket: WebSocket chạy trên TCP nên thừa hưởng nguyên vẹn head-of-line blocking, cộng thêm chi phí bắt tay HTTP - một ràng buộc chỉ cần cho trình duyệt, không cần cho client desktop.

1.4. Kiến trúc phân lớp hệ thống

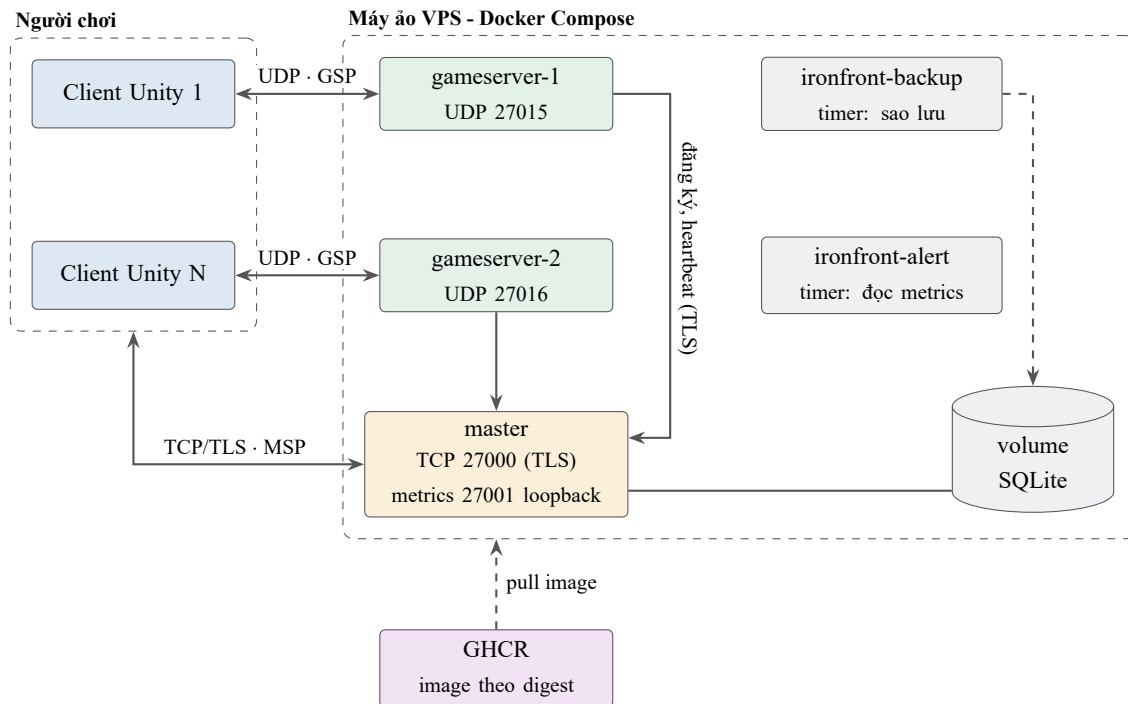
Mã nguồn chia thành các thư viện dùng chung độc lập hoàn toàn với engine đồ hoạ; dự án Unity tích hợp chúng dưới dạng các thư viện liên kết động (DLL). Nhờ vậy logic mạng được kiểm thử tự động độc lập nhanh chóng mà không cần mở Unity Editor.



Hình 1.2. Phân lớp kiến trúc và quan hệ phụ thuộc giữa các phân hệ

Phân hệ / Thành phần	Loại	Quy mô	Vai trò nghiệp vụ
Giao thức chung (Protocol)	Thư viện dùng chung	5 300 dòng	Định dạng gói tin, chuẩn tuần tự hoá và hằng số giao thức
Tầng Giao vận (Transport)	Thư viện giao vận	4 100 dòng	Độ tin cậy trên UDP, kênh truyền, phân mảnh, kiểm soát nghẽn
Đồng bộ và Mô phỏng	Thư viện logic	18 500 dòng	Snapshot, mã hóa delta, vùng quan tâm, tick server, bù trễ
Máy chủ trung tâm (Master)	Ứng dụng dịch vụ	4 300 dòng	Tiếp nhận TCP, xác thực, quản lý phòng, ghép trận, bảo mật
Liên kết máy chủ trung tâm	Thư viện kết nối	1 250 dòng	API trao đổi thông tin với máy chủ trung tâm
Tích hợp giao diện client	Tích hợp ứng dụng	14 600 dòng	Điều phối luồng, dự đoán, nội suy, giao diện sảnh và HUD

1.5. Kiến trúc triển khai



Hình 1.3. Sơ đồ triển khai trên một máy ảo: một master và hai game server headless

Image được triển khai theo bản dựng cố định; khóa bí mật dùng chung và chứng chỉ TLS được cấu hình qua biến môi trường bảo mật độc lập. Sao lưu và cảnh báo chạy bằng bộ định thời hệ thống *ngoài* container, vì bộ phát hiện sự cố phải nằm ngoài thứ nó giám sát. Hạ tầng còn có cấu hình tự động hoá máy ảo và phương án triển khai đám mây thay thế.

1.6. Phân công theo phân hệ

Sinh viên	Mã SV	Phân hệ	Phạm vi nhiệm vụ chính
Nguyễn Thành Trung	B23DCCN861	Client Unity	Giao diện sảnh, HUD, dự đoán và nội suy chuyển động client
Nguyễn Minh Toàn	B23DCCN832	Tầng giao vận UDP	Giao thức UDP tin cậy, kênh truyền, phân mảnh, ACK bitfield
Nguyễn Duy Nghĩa	B23DCCN600	Replication, mô phỏng server	Vòng tick mô phỏng, đồng bộ snapshot, nén delta và bù trễ
Nguyễn Tự Kiên	B23DCCN465	Master Server TCP, hạ tầng	Máy chủ TCP, xác thực, quản lý phòng, ghép trận và hạ tầng VPS

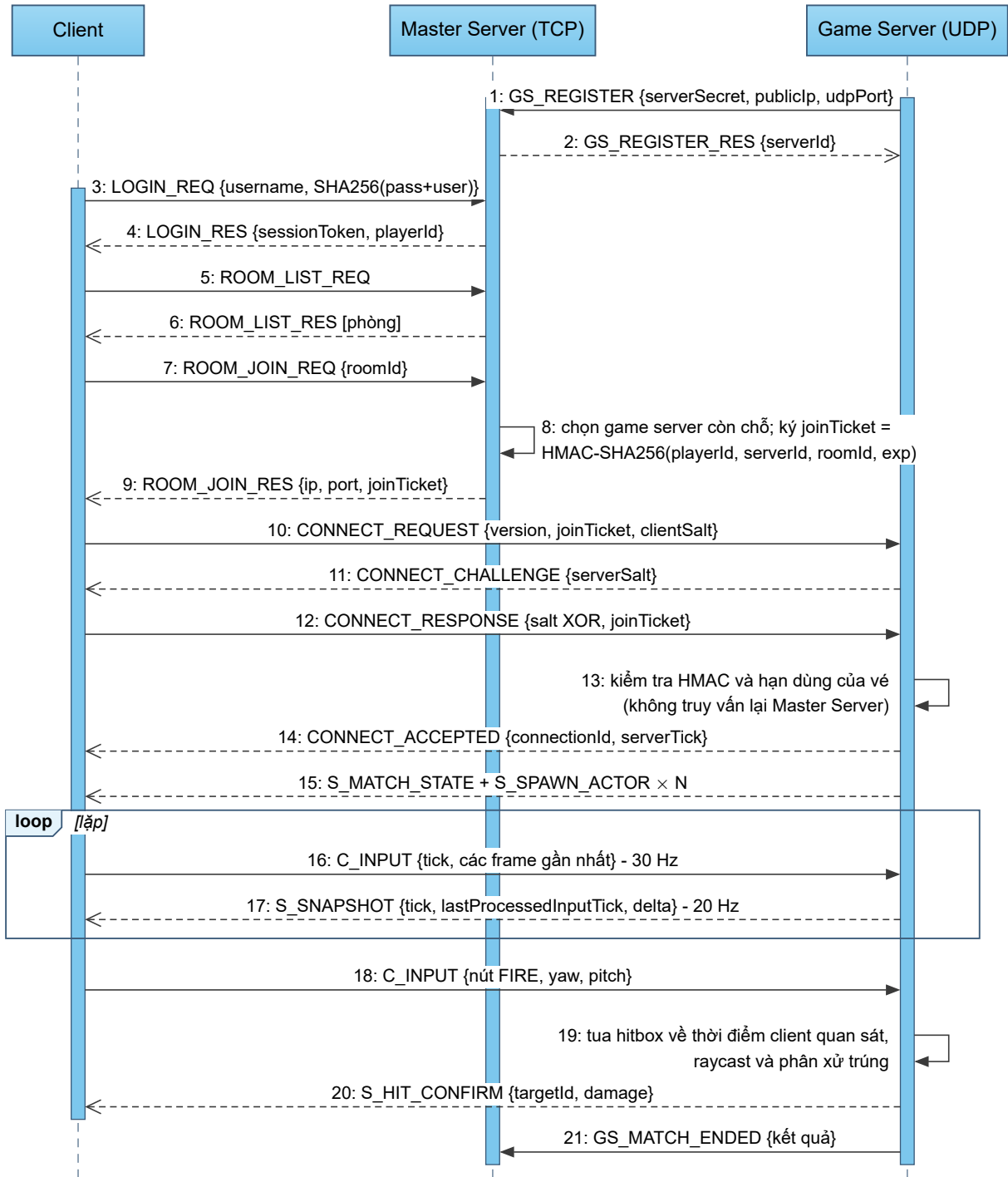
2. Mô tả thiết kế chung của hệ thống

2.1. Mô hình thẩm quyền

Hệ thống **hoàn toàn do server quyết định**: client chỉ gửi *ý định* (input), không bao giờ gửi kết quả. Nhờ vậy gian lận tốc độ, dịch chuyển tức thời, bắn nhanh hay đạn vô hạn trở nên bất khả thi về cấu trúc thay vì chỉ bị phát hiện.

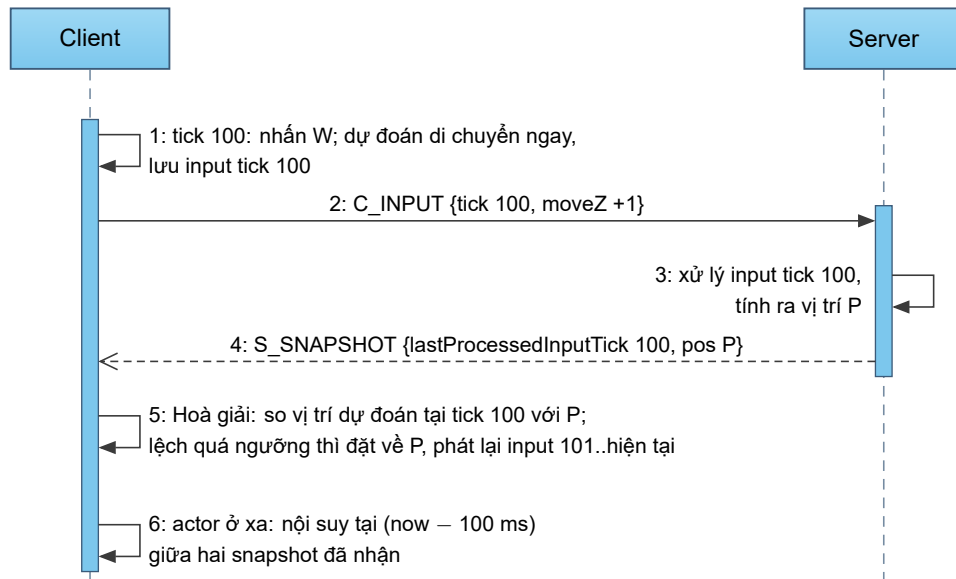
Vấn đề	AI quyết định	Client được làm gì
Vị trí nhân vật	Server	Dự đoán ngay, sau đó hoà giải với server
Trúng đạn, sát thương	Server	Hiện hiệu ứng dự đoán, chờ S_HIT_CONFIRM
Máu, chết, hồi sinh	Server	Chỉ hiển thị
Độ tàn đạn, giật súng	Server	Giật cục bộ cho cảm giác, không ảnh hưởng viên đạn thật
Bot AI, điểm chiếm, điểm số	Server	Nội suy, hiển thị
Lên xuống ghế phương tiện	Server	Gửi yêu cầu C_SEAT_REQUEST
Camera, UI, âm thanh, ragdoll	Client	Toàn quyền, không đồng bộ

2.2. Luồng nghiệp vụ đầu - cuối



Hình 2.1. Luồng đầu - cuối: đăng nhập, vào phòng, kết nối game server; chơi, kết thúc trận

2.3. Các kỹ thuật netcode cốt lõi

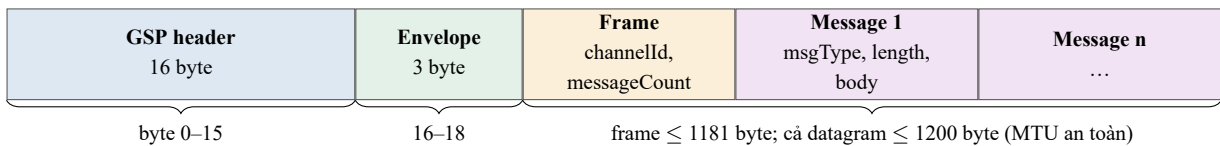


Hình 2.2. Dự đoán phía client, hoà giải với server và nội suy actor ở xa

Kỹ thuật	Mục đích	Tham số
Dự đoán phía client	Người chơi cục bộ di chuyển ngay khi nhấn phím	Cùng thuật toán mô phỏng chuyển động ở client và server
Hoà giải	Sửa sai khi dự đoán lệch, phát lại input chưa xác nhận	Ngưỡng lệch 0,25 m
Nội suy thực thể	Actor ở xa chuyển động mượt dù snapshot chỉ 20 Hz	Bộ đệm 100 ms
Bù trễ	Người ping cao vẫn bắn trúng thứ họ thấy	Tua tối đa 200 ms, lịch sử hitbox 1 s

2.4. Thiết kế giao thức

2.4.1. GSP - cấu trúc một datagram PAYLOAD



Hình 2.3. Ba lớp của một datagram PAYLOAD: header GSP, envelope kênh của transport, frame message của ứng dụng

Kênh	Kiểu	Dữ liệu	Khi mất gói
0	Không tin cậy, không thứ tự	Ping/pong	Bỏ qua
1	Không tin cậy, có thứ tự (server → client)	S_SNAPSHOT, S_WEAPON_FIRE, S_VEHICLE_SNAPSHOT	Bỏ qua; gói cũ bị hủy
2	Tin cậy, có thứ tự	Sự kiện gameplay, spawn/despawn, chat, C_ACK_BASELINE	Gửi lại tới khi được ack
3	Không tin cậy, có thứ tự (client → server)	C_INPUT, C_VEHICLE_INPUT	Bỏ qua; bù bằng dư thừa

Độ tin cậy áp dụng **theo kênh**, không theo kết nối: một sự kiện bị mất trên kênh 2 không làm nguyền snapshot trên kênh 1 - ưu điểm cốt lõi so với TCP vốn ép mọi thứ vào một luồng.

2.4.2. MSP - khung tin trên TCP

length u32, big-endian	msgType u16, little-endian	body UTF-8 JSON, tối đa 64 KB
----------------------------------	--------------------------------------	---

Hình 2.4. Khung tin MSP: tiền tố độ dài giải quyết bài toán ranh giới thông điệp trên luồng byte TCP

2.4.3. joinTicket - cầu nối TCP và UDP

playerId u32	serverId u16	roomId u16	expiresAtUnixMs u64 (60 s)	displayName 16 byte UTF-8	hmac HMAC-SHA256, 32 byte
32 byte dữ liệu được ký					32 byte chữ ký

Hình 2.5. Cấu trúc joinTicket 64 byte

Master ký vé bằng khoá bí mật; game server kiểm tra chữ ký bằng thuật toán so sánh thời gian hằng số và kiểm hạn dùng. Không cần gọi lại master, nên **master có thể dừng giữa trận mà trận vẫn tiếp tục**.

2.5. Tham số thời gian

Tham số	Giá trị	Ghi chú
Tick mô phỏng server	30 Hz (33,3 ms)	Theo đồng hồ thực, không theo chu kỳ cập nhật vật lý
Snapshot	20 Hz (50 ms)	Rẻ hơn 30 Hz, nội suy bù độ mượt
Gửi input	30 Hz	Mỗi gói lặp lại các frame gần nhất
Bộ đệm nội suy	100 ms	Chịu được mất một snapshot
Cửa sổ bù trễ / lịch sử hitbox	200 ms / 1 s	Chặn lạm dụng ping giả
Keep-alive / timeout UDP	1 s / 10 s	
Heartbeat MSP client / game server	15 s	Phát hiện kết nối TCP nửa mở

2.6. Phân loại đối tượng đồng bộ

Loại	Ví dụ	Cách xử lý
Actor được đồng bộ	Người chơi, bot	Có actorId u16, có trong snapshot, mã hoá delta
Phương tiện	Xe, trực thăng, tàu	Luồng riêng S_VEHICLE_SNAPSHOT
Chỉ ở server	Lịch sử hitbox, dữ liệu AI	Không bao giờ gửi
Chỉ ở client	Camera, UI, particle, ragdoll, âm thanh	Không bao giờ gửi
Sự kiện một lần	Chết, nổ, phát súng, chiếm điểm	Kênh tin cậy có thứ tự
Tĩnh	Địa hình, công trình, điểm hồi sinh	Có sẵn trong scene hai phía

2.7. Bảo mật ở mức phù hợp phạm vi

Mối đe dọa	Biện pháp	Tầng
Tăng tốc, dịch chuyển, bắn nhanh, đạn vô hạn	Server chỉ nhận input; kẹp tốc độ; tự quản lý cooldown và đạn	Replication
Mạo danh người chơi	connectionId gắn với playerId trong joinTicket đã ký	Transport
Khuếch đại giả mạo IP	Bắt tay challenge không lưu trạng thái (cookie HMAC)	Transport
Gói rác, lụt gói	Kiểm protocolId; giới hạn tốc độ theo IP; giới hạn nhóm phân mảnh	Transport
Nghe lén mật khẩu	TLS và ghim vân tay chứng chỉ; SHA-256 phía client, bcrypt phía server	Master
Tràn bộ nhớ, Slowloris	Khung tối đa 64 KB; hạn 30 s trước xác thực; 5 kết nối/IP	Master
SQL injection, dò mật khẩu	Truy vấn tham số hoá; 5 lần/phút/IP; khoá 15 phút sau 10 lần sai	Master

2.8. Các quyết định kiến trúc đã chốt

Mã	Quyết định	Lý do
AD-1	Server nắm quyền, không có listen-server	Thảm quyền rõ ràng; tránh sửa 21 singleton
AD-2	Game server là Unity headless	Tái dùng PhysX, A*, AI; tiết kiệm 8-12 tuần
AD-3	Không theo đuổi tắt định (lockstep)	Bộ sinh ngẫu nhiên rải rác; engine vật lý không tắt định
AD-4	Ragdoll chỉ là hiệu ứng cục bộ	Tiết kiệm khoảng 1,7 MB/s
AD-5	Transport, Replication là thư viện .NET thuần	Kiểm thử bằng xUnit, tránh xung đột Unity
AD-7	Snapshot không tin cậy, sự kiện tin cậy	Đúng bản chất trạng thái và sự kiện
AD-8	Không dùng WebSocket	Yêu cầu môn học và head-of-line blocking

2.9. Quy trình phát triển và bảo đảm chất lượng

- **Đặc tả giao thức đóng băng** là nguồn sự thật duy nhất của hệ thống; công cụ kiểm tra tự động sẽ cảnh báo nếu cấu trúc gói tin của các bên không đồng nhất.
- **Tách người thiết kế và người kiểm thử:** nhóm giao vận xây dựng cơ chế đóng gói dữ liệu, nhóm đồng bộ xây dựng bộ kiểm thử tính tương thích theo đặc tả.
- **Tích hợp liên tục (CI)** chạy tự động: kiểm tra toàn diện, coi cảnh báo là lỗi, chạy toàn bộ bộ kịch bản kiểm thử.

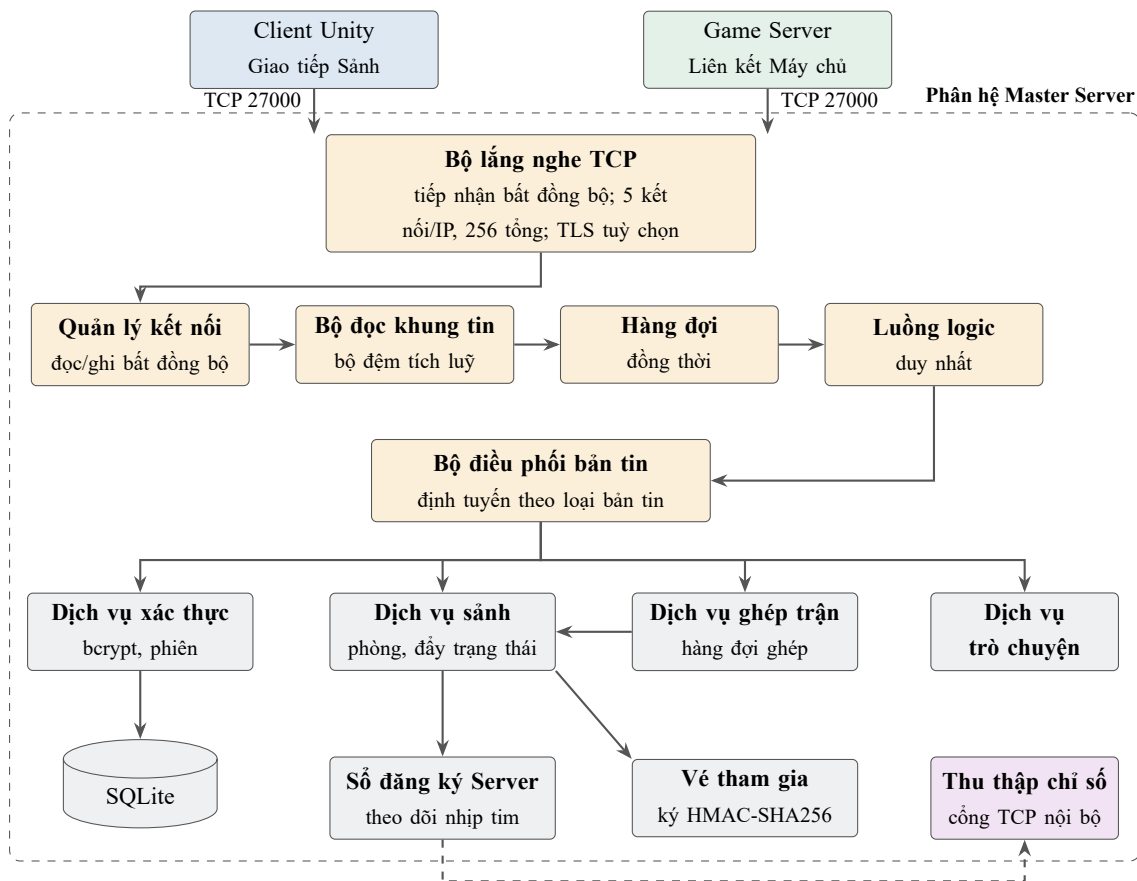
- **Tiêu chuẩn hoàn thành (Definition of Done):** hoàn thành kiểm thử thực tế, toàn bộ kiểm nghiệm tự động đạt chuẩn, ứng dụng biên dịch sạch sẽ, tài liệu kỹ thuật hoàn thiện.

II. NỘI DUNG CÁ NHÂN - Phân hệ Master Server TCP và hạ tầng.

3. Biểu đồ và giao diện nội dung cá nhân thực hiện

Phân hệ Master Server là **máy chủ TCP thuần .NET 8** - không ASP.NET Core, không SignalR, không WebSocket - chỉ sử dụng tầng Socket mạng nền tảng và cơ chế lắng nghe kết nối tiêu chuẩn của hệ điều hành. Trọng tâm là bài toán **đóng khung thông điệp trên luồng byte TCP**. Phạm vi còn gồm module thư viện cho client Unity, liên kết game server - master, công cụ kiểm thử tải và hạ tầng triển khai (Docker, systemd, TLS, Terraform).

3.1. Kiến trúc thành phần Master Server



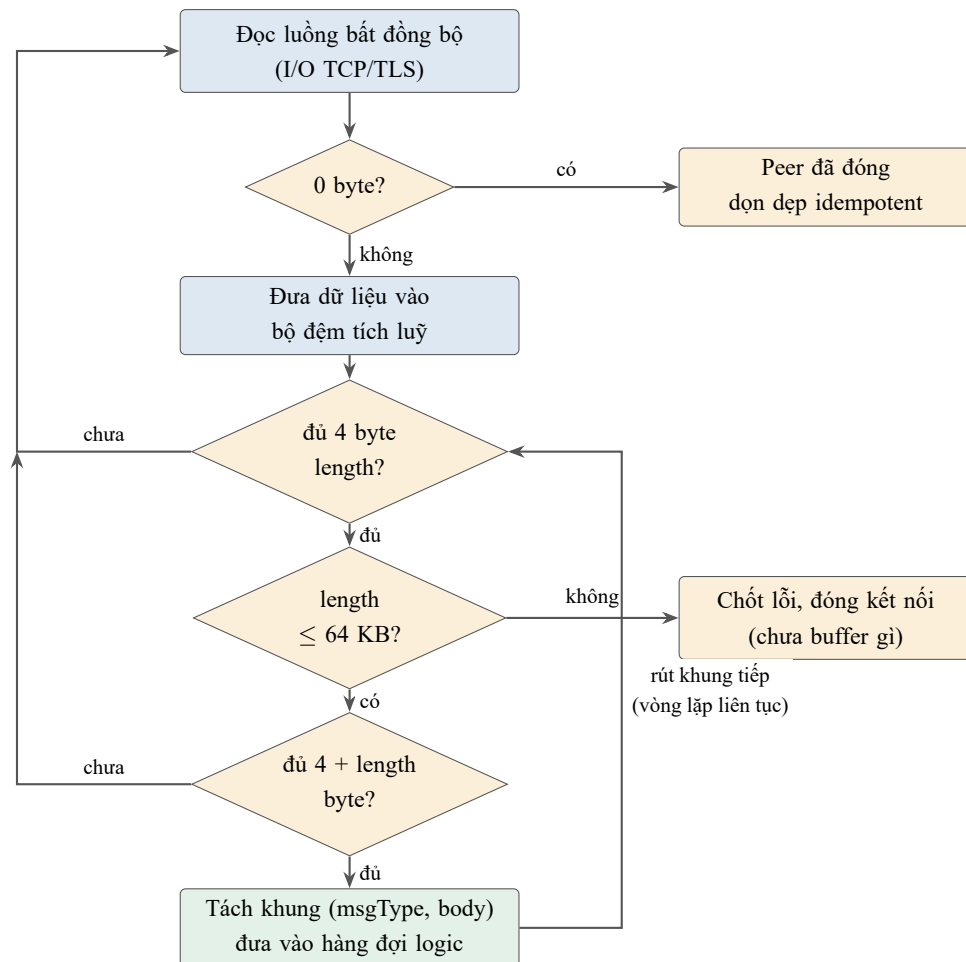
Hình 3.1. Biểu đồ thành phần Master Server: I/O bất đồng bộ trên thread pool, logic trên một luồng duy nhất

3.2. Đóng khung tin trên luồng TCP

length - u32 big-endian 00 00 00 09	msgType - u16 little-endian 01 00 = LOGIN_REQ	body - UTF-8 JSON 7B 22 75 22 3A 31 7D = {"u":1}
---	---	--

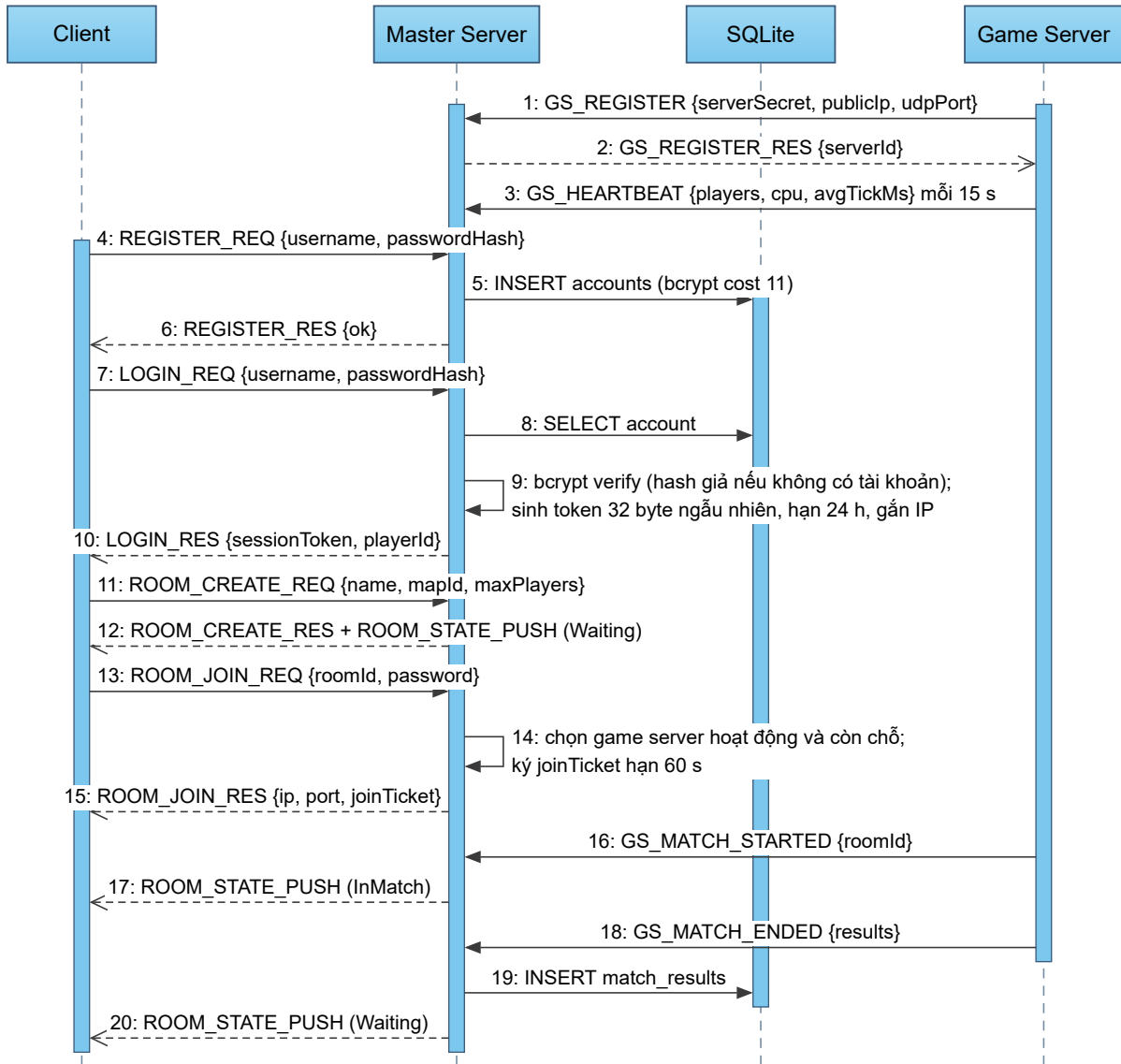
Hình 3.2. Khung tin MSP tham chiếu 13 byte (được ghim trong bộ test conformance)

Trường hợp đọc	Điều xảy ra nếu xử lý sai	Số liệu đo thực tế
Một lần đọc, một khung	Không lỗi - trường hợp ai cũng thử	Chỉ trùng khớp do tình cờ
Dính (glue)	Khung 2..n bị bỏ hoặc kết nối treo	1000 lần Send → 942 lần Receive
Tách (split)	Không bao giờ ghép đủ khung, đọc rác từ tiền tố cụt	1 khung 58 KB → 8 lần Receive
Dính + phần dư đuôi	Cả hai lỗi cùng lúc; phải giữ phần đuôi qua các lần đọc	Kiểm thử riêng



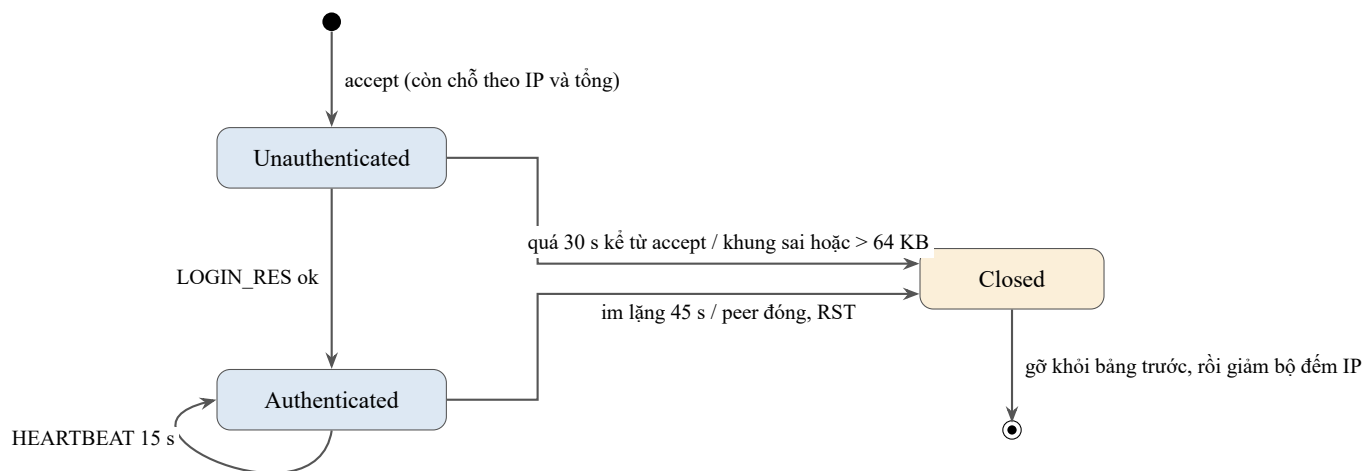
Hình 3.3. Thuật toán bộ đọc tích lũy: rút mọi khung hoàn chỉnh trong vòng lặp quét cạn, không kiểm tra một lần

3.3. Luồng dịch vụ sảnh chờ

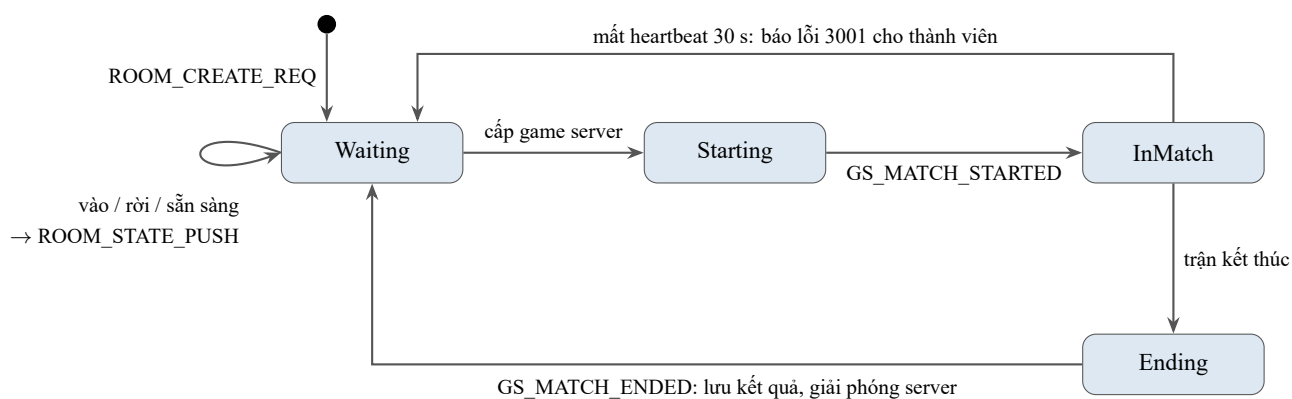


Hình 3.4. Biểu đồ tuần tự dịch vụ sảnh: đăng ký game server, tài khoản, phòng, joinTicket, kết quả trận

3.4. Vòng đời kết nối và phòng



Hình 3.5. Vòng đời một kết nối TCP: hạn tuyệt đối trước xác thực, hạn im lặng sau xác thực



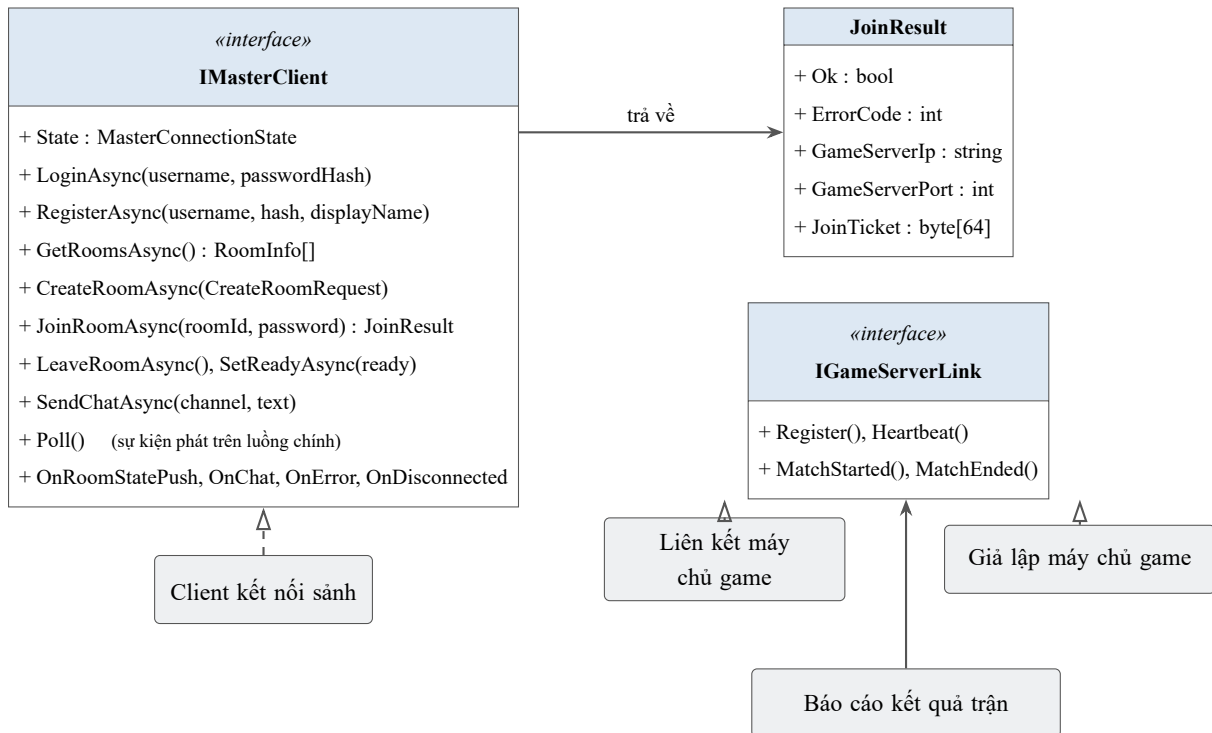
Hình 3.6. Vòng đời phòng (RoomLifecycleState)

3.5. Mô hình dữ liệu

accounts		
INTEGER	player_id	PK
TEXT	username	UNIQUE, NOCASE
TEXT	password_hash	bcrypt
TEXT	display_name	
INTEGER	created_at	
INTEGER	last_login_at	
INTEGER	failed_logins	
INTEGER	locked_until	
INTEGER	is_banned	

Hình 3.7. Lược đồ SQLite của Master Server (phiên và phòng được giữ trong bộ nhớ)

3.6. Giao diện lập trình và giao diện giám sát

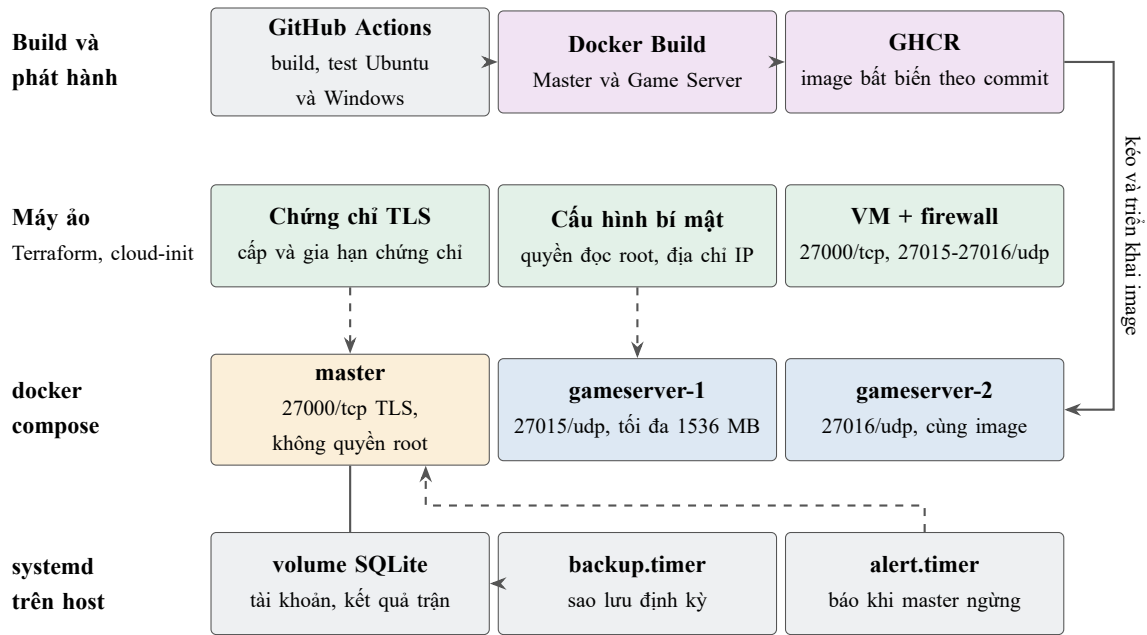


Hình 3.8. Giao diện lập trình cho Client và liên kết Game Server; mọi sự kiện nghiệp vụ được đồng bộ về luồng chính

Chỉ số giám sát dịch vụ (Công TCP Metrics)		Kết quả kiểm thử tải áp lực (Load Test)				
Chỉ số giám sát	Giá trị đo	Kịch bản đo	Tải/giây	p50	p99	Lỗi
Thời gian hoạt động	5 412 giây	Hành vi ngẫu nhiên	143,9	0,81 ms	5,77 ms	0
Kết nối hiện thời / đỉnh	18 / 32 kết nối	Quét danh sách phòng	12 230,0	0,89 ms	7,42 ms	0
Tổng kết nối đã nhận	1 411 kết nối	Vào và rời phòng liên tục	130,4	0,72 ms	5,68 ms	147
Tổng khung tin đã nhận	88 213 khung	32 máy khách song song	206,9	1,03 ms	8,51 ms	0
Lỗi bắt tay bảo mật TLS	0 lỗi	Bộ nhớ RAM: 64-74 MB khi chạy thông thường; duy trì 1 000 kết nối đồng thời chiếm 81 MB.				
Tài khoản đang trực tuyến	16 / 57 tài khoản					
Phòng hoạt động / vào trận	3 / 2 phòng					
Máy chủ trận khả dụng	2 / 2 máy chủ					
Tỷ lệ đăng nhập / phút	4 lượt / phút					

Hình 3.9. Giao diện giám sát thời gian thực và kết quả kiểm thử chịu tải của phân hệ Master Server

3.7. Hạ tầng triển khai



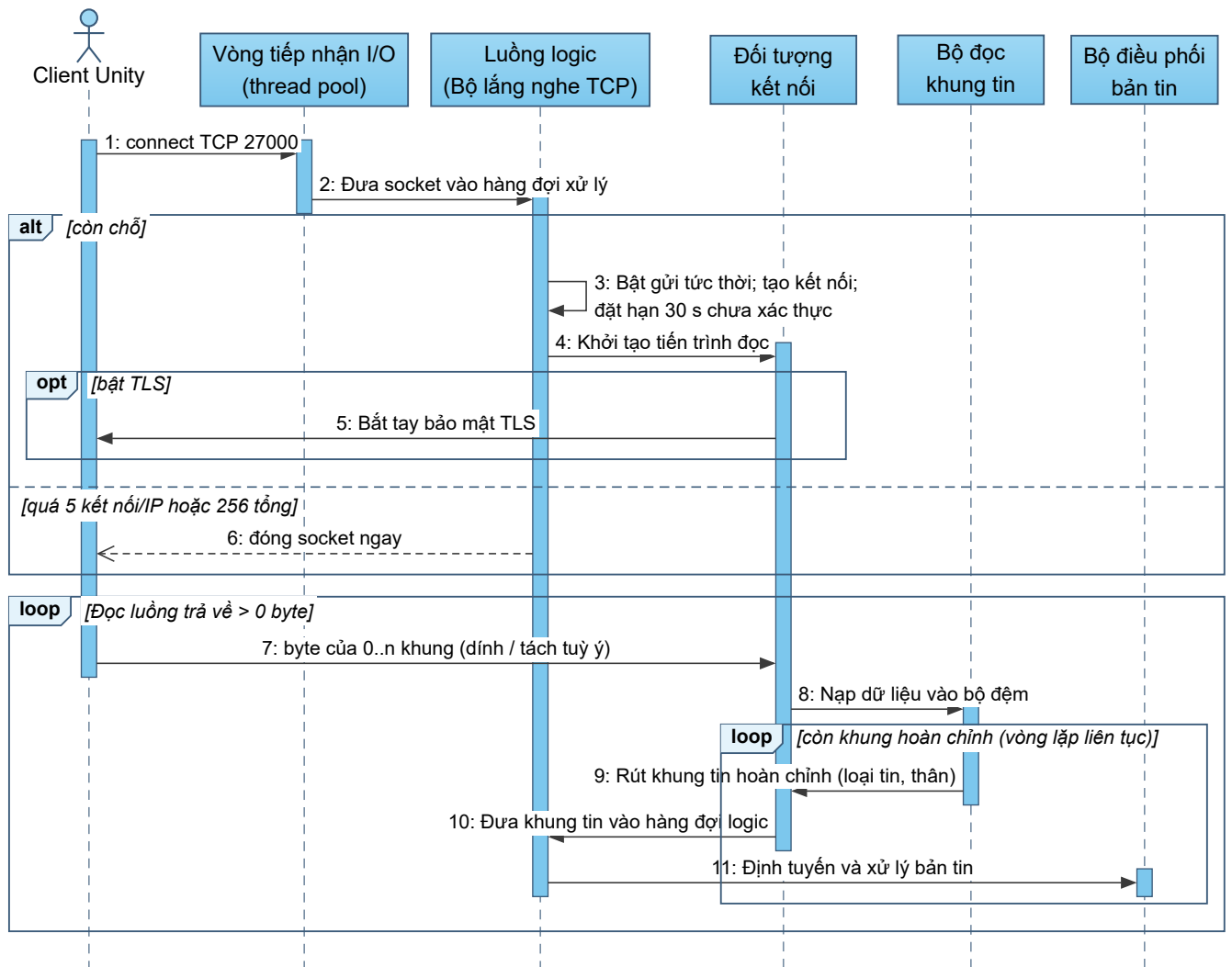
Hình 3.10. Đường ống triển khai và topo hạ tầng trên VPS (fly.io là phương án thay thế)

4. Luồng xử lý và giải pháp đã ứng dụng

Mục này trình bày chi tiết từng luồng nghiệp vụ mà bản mô tả đề tài giao cho phân hệ: kết nối TCP nhiều client, khung bản tin nhị phân, gửi nhận trên luồng nền phía client, đăng ký - đăng nhập - khoá tài khoản, danh sách phòng, tạo và vào phòng, ghép trận nhanh, chat sánh, cấp vé vào trận, đăng ký và báo sống của game server, lưu kết quả trận, bảo mật kênh TCP và triển khai. Mỗi luồng gồm sơ đồ tuần tự, diễn giải nghiệp vụ và bảng **vấn đề - giải pháp**.

4.1. Luồng tiếp nhận kết nối và đọc khung tin

Mô hình luồng. Mọi thao tác I/O (tiếp nhận kết nối, đọc, ghi luồng byte) chạy bất đồng bộ trên thread pool và không bao giờ chặn. Mọi **logic** (nhận hay từ chối kết nối, xử lý khung, cập nhật phiên và danh sách phòng) chạy trên **đúng một luồng**, được cấp việc qua một hàng đợi đồng thời. Vì chỉ có một luồng chạm vào dữ liệu nghiệp vụ nên không cần khoá luồng (lock-free logic), triệt tiêu hoàn toàn nguy cơ tranh chấp tài nguyên sánh chờ.

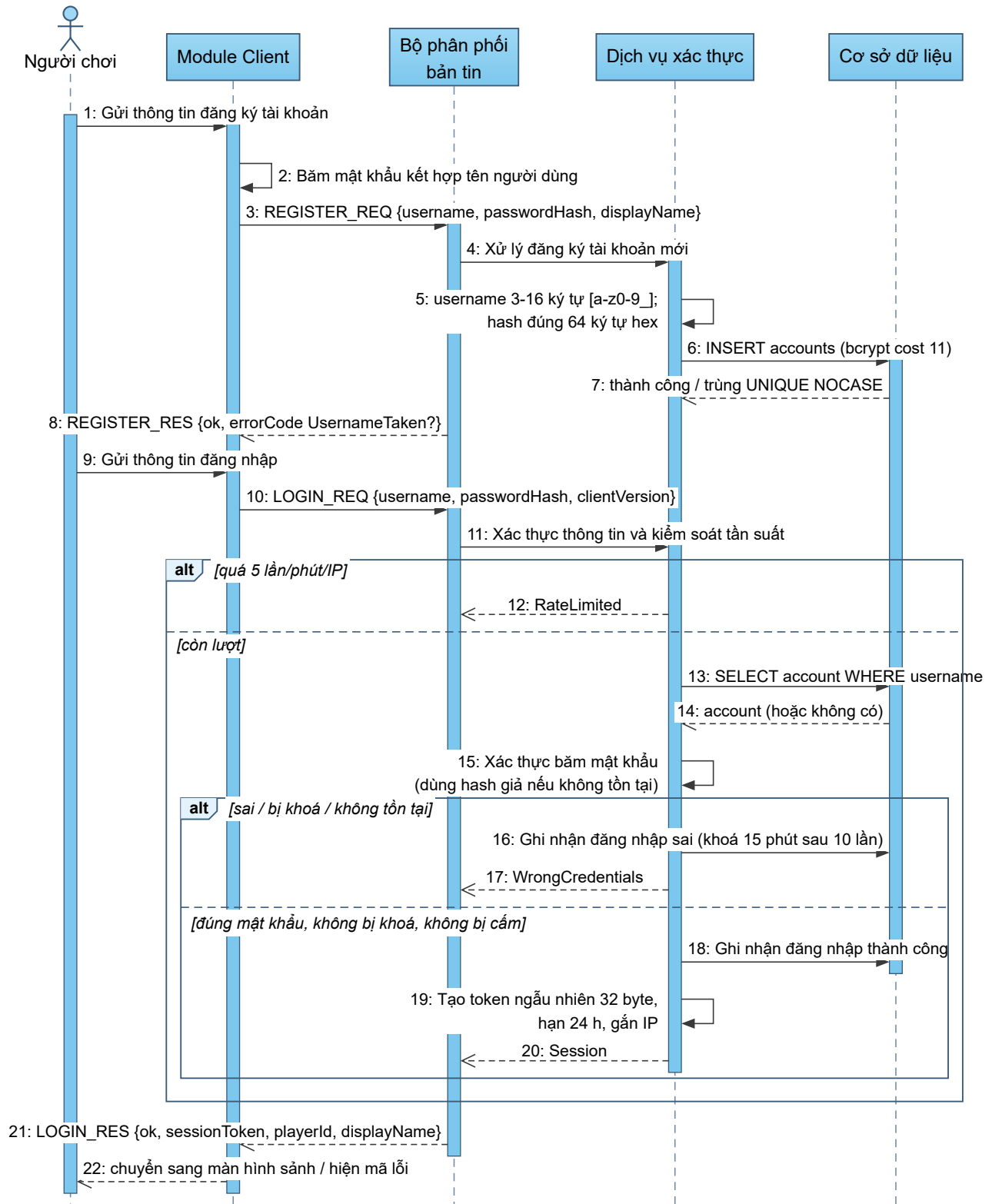


Hình 4.1. Biểu đồ tuần tự tiếp nhận kết nối, bắt tay TLS và rút khung tin từ luồng byte TCP

Vấn đề	Giải pháp đã áp dụng
TCP là luồng byte, không có ranh giới bản tin	Khung tiền tố độ dài: <code>length u32 big-endian + msgType u16 + thân JSON</code> ; bộ đọc tích lũy giữ phần đuôi dở dang qua các lần đọc và rút khung bằng vòng lặp quét cạn (Module đọc khung tin nhị phân)
Kích thước <code>length</code> do client điều khiển (nguy cơ cấp phát bộ nhớ ảo khổng lồ)	Kiểm tra giới hạn an toàn 64 KB trước khi đệm dữ liệu; vi phạm thì bộ đọc lập tức chốt lỗi và đóng kết nối ngay (Bộ đọc khung tin)
Tấn công Slowloris: giữ kết nối nhỏ giọt từng byte để chiếm tài nguyên	Chưa đăng nhập: áp đặt hạn tuyệt đối 30 s kể từ lúc tiếp nhận (không gia hạn khi có byte mới); đã đăng nhập: hạn im lặng 45 s (Cấu hình tham số máy chủ kết nối)
Một địa chỉ IP hoặc botnet chiếm dụng toàn bộ tài nguyên	Thiết lập giới hạn tối đa 5 kết nối/IP và 256 kết nối toàn hệ thống (Bộ tiếp nhận kết nối)
Thuật toán Nagle và độ trễ ACK làm trễ các bản tin nhỏ	Kích hoạt cờ gửi tức thì không đệm (tắt thuật toán Nagle); mỗi khung dữ liệu được gửi đi trong một thao tác mạng duy nhất (Bộ tiếp nhận kết nối và quản lý kênh truyền)
Kết nối nửa mở (ngắt kết nối bất ngờ, rút cáp mạng không phát sinh FIN/RST)	Bản tin nhịp tim mỗi 15 s từ client; sau 45 s không có tín hiệu thì đóng kết nối. Kiểm nghiệm thực tế với tín hiệu đặt lại cường bức: 1 376 lần ngắt đột ngột trong 30 s đều được thu hồi hoàn toàn (Kịch bản kiểm thử ngắt kết nối đột ngột)
Thao tác dọn dẹp chạy đồng thời gây sai lệch bộ đếm kết nối	Gỡ bản ghi khỏi bảng kết nối trước , chỉ giảm bộ đếm IP nếu thao tác gỡ diễn ra thành công (tính chất luồng đơn và idempotent) (Bộ quản lý kết nối)

Phía client. Vòng nhận chạy trên luồng nền; mỗi khung nhận được được chép và đưa vào hàng đợi luồng an toàn. Chu kỳ cập nhật game gọi lệnh rút hàng đợi mỗi khung hình trên luồng chính, nên mọi sự kiện giao diện và nghiệp vụ được phát trên luồng chính - giao diện không bao giờ bị treo khi chờ mạng và logic xử lý không bị vi phạm ngưỡng luồng. Mỗi yêu cầu bất đồng bộ được liên kết với một cơ chế phản hồi hoàn thành khi nhận được gói tin trả lời từ máy chủ.

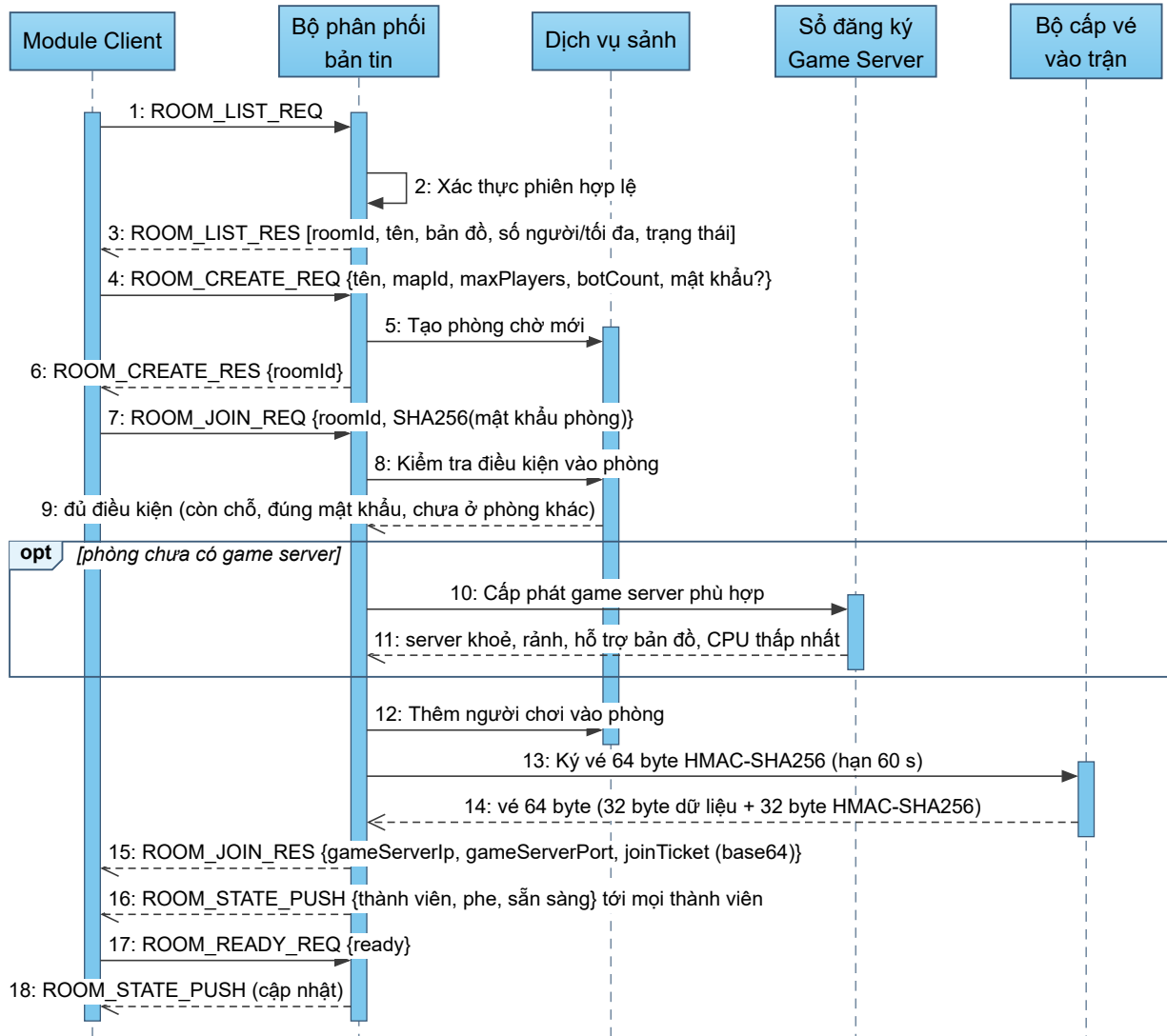
4.2. Luồng đăng ký và đăng nhập



Hình 4.2. Biểu đồ tuần tự đăng ký và đăng nhập, gồm giới hạn tần suất, khoá tạm thời và chống dò tài khoản

Yêu cầu trong đề / vấn đề	Giải pháp đã áp dụng
Mật khẩu phải được hash trước khi lưu	Hai lớp: client gửi SHA256(mật khẩu + username) (server không bao giờ thấy mật khẩu gốc); server lưu bcrypt(hash, cost 11) (lộ CSDL vẫn khó dò ngược)
Username đã tồn tại, không phân biệt hoa thường	Username chỉ gồm chữ thường, số và gạch dưới (3-16 ký tự); cột username UNIQUE COLLATE NOCASE và truy vấn không phân biệt chữ hoa chữ thường; chèn trùng trả mã lỗi tên đã tồn tại thay vì làm gián đoạn hệ thống
Sai mật khẩu nhiều lần liên tiếp thì khoá tạm thời	Bộ đếm lần đăng nhập sai tăng mỗi lần sai; tới 10 lần thì khoá tài khoản 15 phút; đăng nhập đúng thì đặt lại bộ đếm
Dò mật khẩu từ một địa chỉ	Tối đa 5 lần đăng nhập/phút/IP, vượt ngưỡng trả mã lỗi giới hạn tần suất
Dò tên tài khoản qua thông báo hoặc qua thời gian phản hồi	Cùng một mã lỗi cho “không có tài khoản” và “sai mật khẩu”; khi không có tài khoản vẫn chạy hàm băm mật khẩu chuẩn với một chuỗi băm giả cố định để thời gian phản hồi như nhau (tránh tấn công đo thời gian chênh lệch khoảng 190 ms)
Chiếm đoạt phiên làm việc	Token ngẫu nhiên an toàn 32 byte sinh từ bộ sinh số ngẫu nhiên mật mã học, hạn 24 h, gắn với IP; không bao giờ ghi token vào log
Tấn công SQL injection	Toàn bộ truy vấn cơ sở dữ liệu dùng tham số ràng buộc chuẩn, tuyệt đối không ghép chuỗi

4.3. Luồng danh sách phòng, tạo phòng, vào phòng và cấp vé



Hình 4.3. Biểu đồ tuần tự sảnh chờ: xem danh sách, tạo phòng, vào phòng, chọn game server và cấp vé có chữ ký

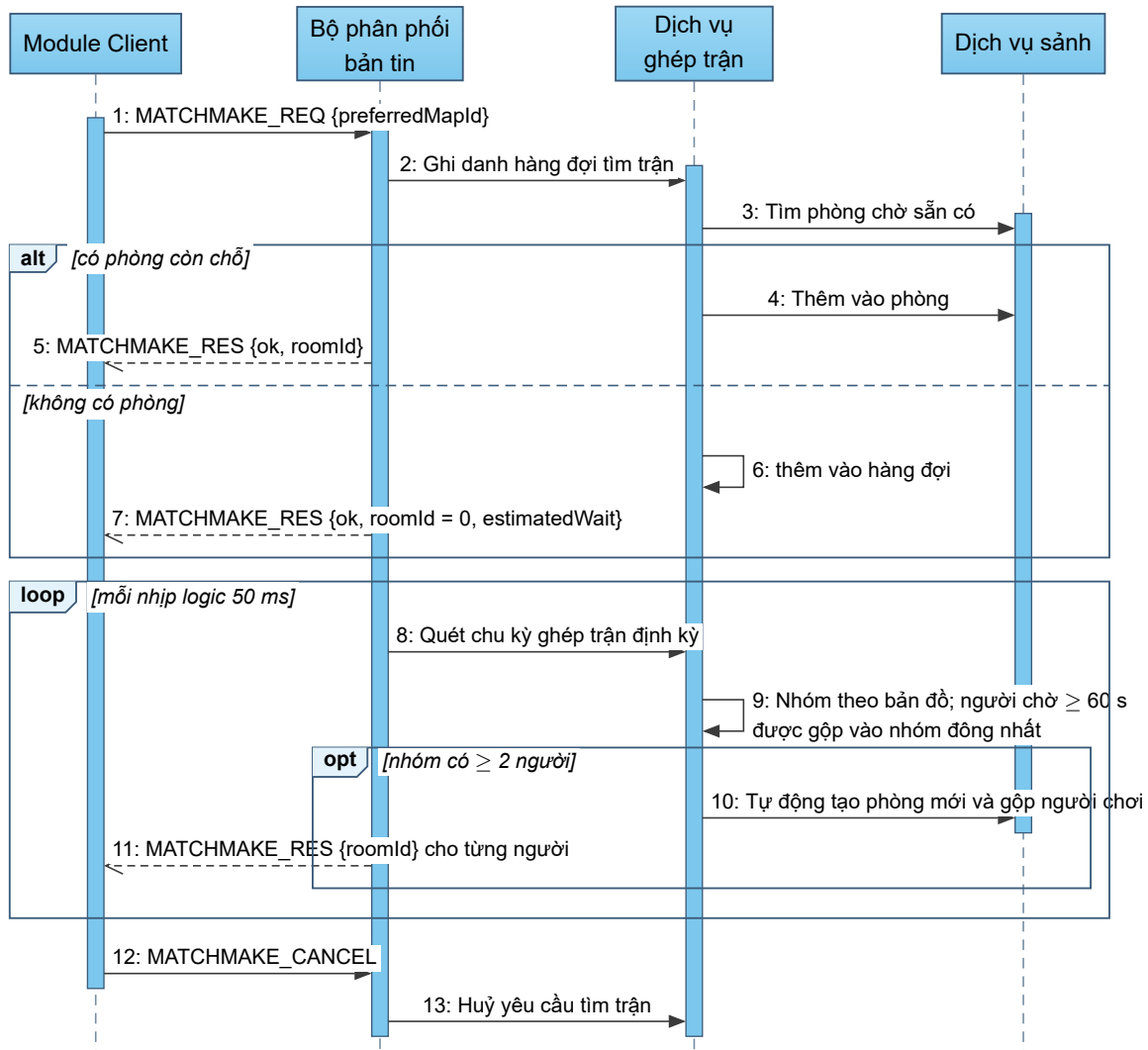
Diễn giải và giải pháp.

- **Đẩy trạng thái thay vì hỏi vòng.** Mỗi khi phòng thay đổi (vào, rời, sẵn sàng), dịch vụ sảnh phát sự kiện phòng thay đổi và bộ phân phối gửi thông báo cập nhật tới mọi thành viên. Một lobby kiểu REST phải hỏi vòng: 16 client hỏi 1 lần/giây là 16 yêu cầu/giây chỉ để báo “không có gì”, và vẫn trễ tới 1 giây.
- **Chọn game server theo sức khỏe, không chỉ theo “còn sống”.** Một server khỏe khi đã báo sống trong 15 s gần nhất, CPU < 90% và thời gian tick trung bình < 40 ms. Trong các server khỏe, rảnh và hỗ trợ bản đồ, chọn server có CPU thấp nhất. Không có server phù hợp thì trả mã lỗi máy chủ bận.
- **Vé vào trận không giả được.** Vé 64 byte gồm thông tin người chơi, mã server, mã phòng, hạn dùng 60 s và tên hiển thị, ký HMAC-SHA256 bằng khoá bí mật chỉ master và game server biết. Game server tự kiểm tra vé bằng thuật toán so sánh thời gian hằng (chống tấn công đo thời gian dò chữ ký từng byte), **không cần gọi lại master**. Hệ quả: master có thể dừng giữa trận mà trận vẫn chạy bình thường.
- **Không ghi vé vào log.** Vé là chứng chỉ mang theo được dùng trong 60 s; log chỉ ghi mã người chơi, mã phòng và mã

server.

- **Hoàn tác khi vào phòng thất bại.** Nếu đã cấp server cho phòng nhưng người đầu tiên vào không thành, server được trả lại registry để phòng không giữ một server không dùng.

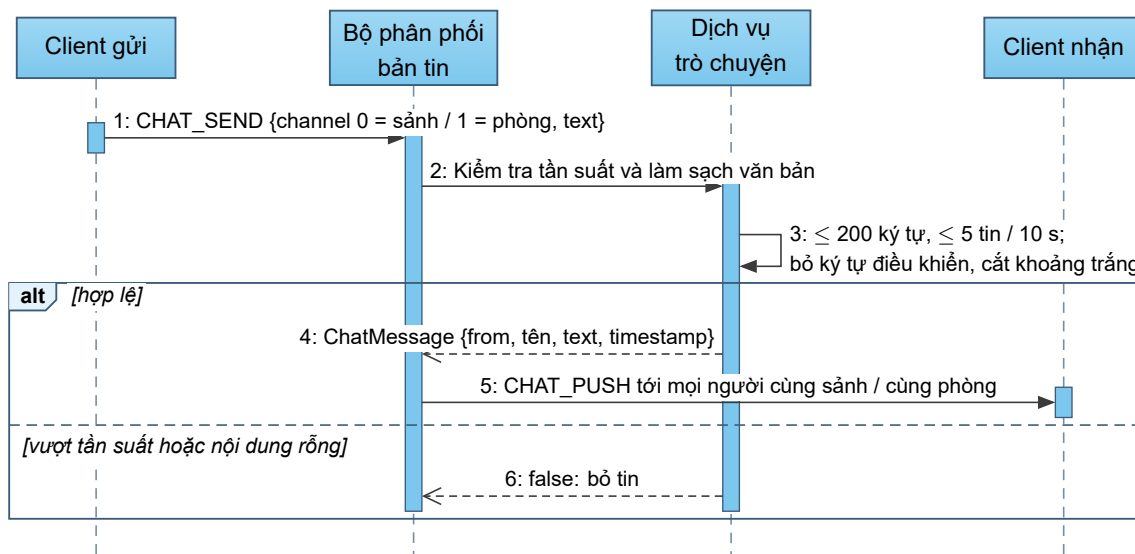
4.4. Luồng ghép trận nhanh



Hình 4.4. Biểu đồ tuần tự ghép trận nhanh: vào phòng sẵn có hoặc chờ trong hàng đợi theo bản đồ

Lỗi đã tìm thấy và cách sửa. Phiên bản đầu “nói điều kiện” sau 60 s bằng cách đổi bản đồ ưu tiên của người chờ lâu thành 0 - thực chất đẩy họ sang một nhóm **riêng**, nên người đã chờ một phút chỉ ghép được với người cũng đã chờ một phút. Người mới vào và người chờ lâu đứng cạnh nhau mà không bao giờ ghép, không lỗi, không log. Cách sửa: gộp người chờ lâu vào **nhóm đồng nhất** hiện có (cơ chế hấp thu hàng đợi nói lỏng); nếu chưa có nhóm nào thì họ tự lập nhóm với bản đồ có id nhỏ nhất.

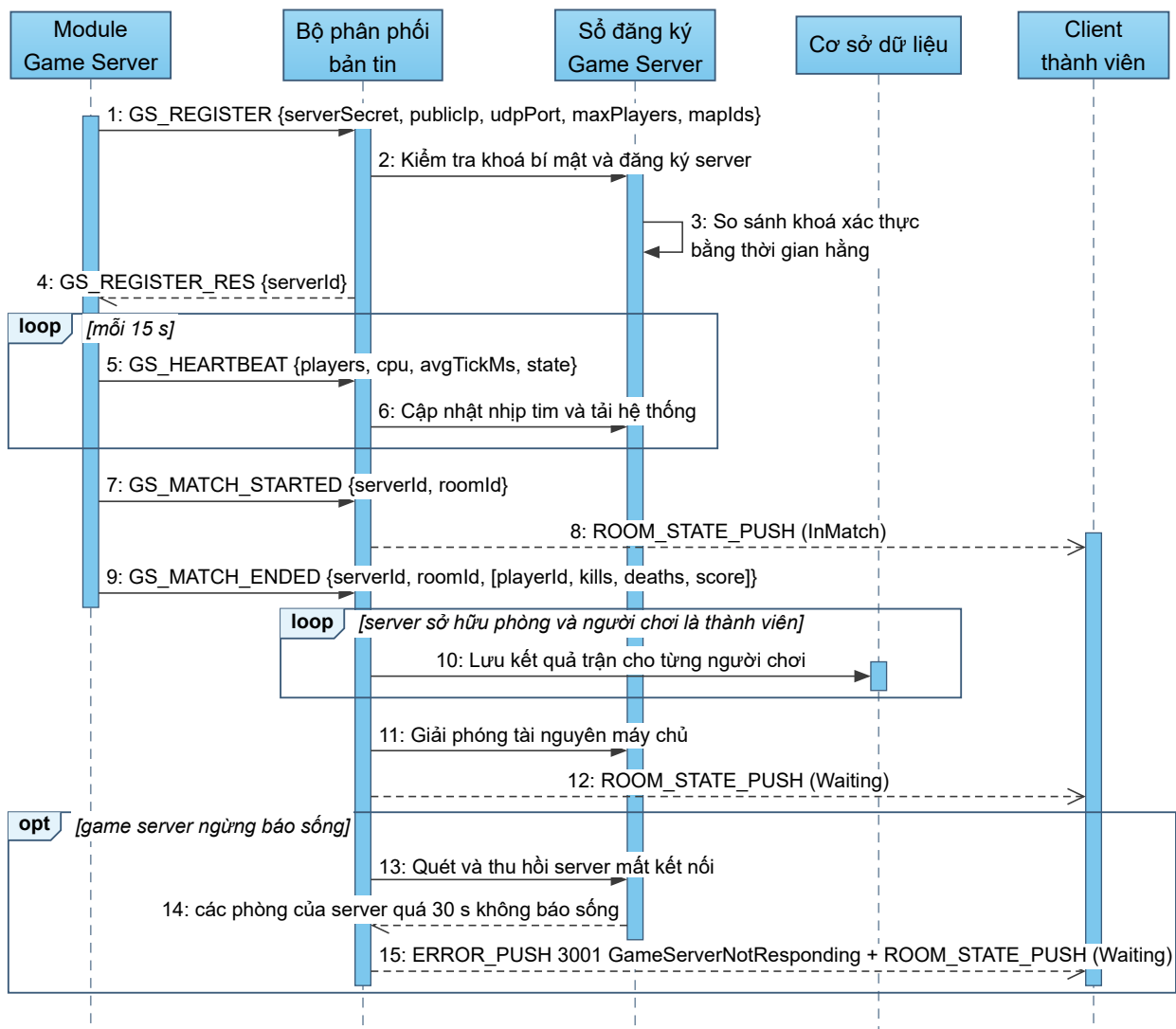
4.5. Luồng chat trong sảnh chờ



Hình 4.5. Biểu đồ tuần tự chat sảnh chờ với giới hạn tần suất và làm sạch nội dung

Tin chat chỉ được chuyển tiếp, không lưu lịch sử. Làm sạch ký tự điều khiển và ký tự định dạng Unicode ngăn một người chơi chèn ký tự đảo chiều văn bản hoặc xuống dòng giả vào giao diện của người khác.

4.6. Luồng game server: đăng ký, báo sống, báo kết quả trận và sự cố



Hình 4.6. Biểu đồ tuần tự vòng đời game server và lưu kết quả trận

Vấn đề	Giải pháp đã áp dụng
Máy lạ giả làm game server	Game server phải gửi đúng khoá bí mật chung của hệ thống (lấy từ biến môi trường, tối thiểu 32 ký tự); registry so sánh bằng thuật toán thời gian hằng. Địa chỉ công khai lấy từ cấu hình khi server chạy trong container sau NAT
Game server chết giữa trận làm phòng kẹt mãi	Bộ dọn 30 s gỡ server ngừng báo sống, trả phòng về trạng thái chờ để cấp lại server khác và báo lỗi 3001 cho mọi thành viên
Server gửi kết quả cho phòng không phải của mình, hoặc chen người ngoài	Cơ chế kiểm tra quyền sở hữu phòng đối chiếu danh danh kết nối; chỉ lưu kết quả của người chơi thực sự là thành viên phòng
Lưu kết quả để xem lại	Bảng dữ liệu kết quả thi đấu có chỉ mục theo mã người chơi để cộng tổng số trận, hạ gục, chết và điểm của một người

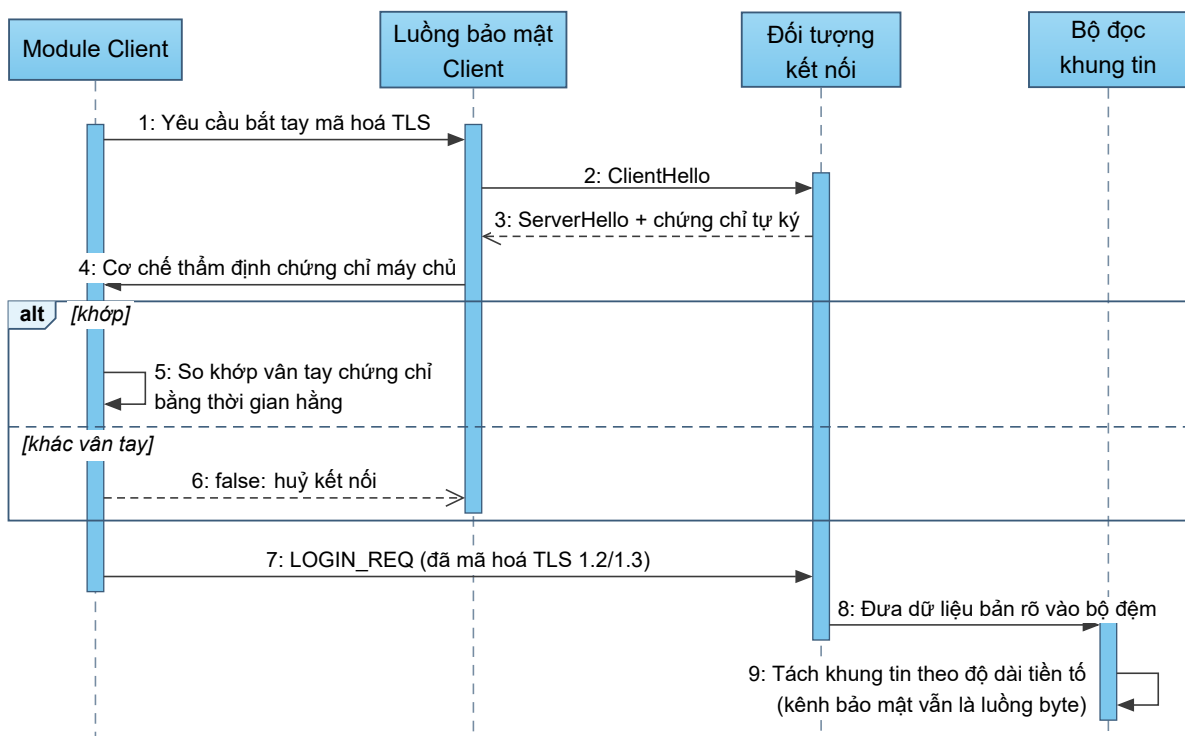
4.7. Luồng ngắt kết nối và dọn dẹp

Khi một kết nối kết thúc vì bất kỳ lý do nào (client đóng, RST, hết hạn im lặng, khung sai), luồng logic gọi bộ xử lý sự kiện ngắt kết nối và dọn dẹp tài nguyên theo thứ tự cố định:

1. Nếu kết nối là của game server: gỡ khỏi sổ đăng ký và đặt lại mọi phòng nó đang giữ (báo lỗi 3001).
2. Gỡ mã người chơi khỏi bảng kết nối, xoá bộ đếm tin nhắn, huỷ vị trí trong hàng đợi ghép trận.
3. Rời phòng; nếu phòng còn người thì gửi thông báo cập nhật phòng cho những người còn lại.
4. Xoá phiên (token) khỏi bộ nhớ RAM.

Phiên được giữ trong bộ nhớ: khởi động lại master sẽ đăng xuất mọi người - chấp nhận có chủ đích để không phải ghi CSDL mỗi lần đăng nhập. Trận đang chạy không bị ảnh hưởng vì game server không cần master sau khi vé đã được kiểm.

4.8. Bảo mật kênh TCP bằng TLS



Hình 4.7. Biểu đồ tuần tự bắt tay TLS với ghim vân tay chứng chỉ

Giải pháp. Máy chủ ảo không có tên miền nên không lấy được chứng chỉ công khai; thay vì hàm kiểm tra luôn trả về hợp lệ (tắt hoàn toàn việc kiểm tra), client được tích hợp sẵn vân tay SHA-256 của đúng một chứng chỉ chỉ định và chỉ chấp nhận chứng chỉ đó - chặt hơn cả chuỗi CA công khai. Nhánh thử nghiệm không kiểm tra chỉ có trong môi trường phát triển nội bộ, bị loại khỏi bản phát hành lúc biên dịch. Kênh mã hoá TLS không làm thay đổi đặc tính luồng byte: kịch bản kiểm tra toàn vẹn luồng dính và tách qua kênh TLS đã chứng minh việc tách khung hoạt động ổn định.

4.9. Tổng hợp kiểm thử và số đo

Kịch bản kiểm thử (theo đề)	Kết quả / cách kiểm
Đăng ký, username trùng, sai mật khẩu, tài khoản bị khoá	Kiểm thử tự động module xác thực và cơ sở dữ liệu; khoá sau 10 lần sai, mở sau 15 phút
Nhiều client cùng kết nối	Kiểm thử tải với 16 và 32 client: p99 5,8 - 8,5 ms, 0 lỗi; duy trì 1 000 kết nối đồng thời với 81 MB RAM
Khung dính, tách, dính + đuôi, khung quá 64 KB	1 000 lần gửi → 942 lần nhận vẫn ra đủ 1 000 khung; 1 khung 58 KB qua 8 lần nhận
Ngắt đột ngột (tín hiệu RST)	1 376 lần ngắt trong 30 s đều được thu hồi, bộ đếm kết nối IP không bị sai lệch
Game server tắt đột ngột	Sau 30 s phòng về trạng thái chờ, thành viên nhận thông báo lỗi 3001
Vé giả / vé hết hạn	Kịch bản từ chối vé bị sửa đổi chữ ký hoặc vé quá hạn từ chối chính xác với mã lỗi quy chuẩn

Bộ kiểm thử tự động của phân hệ Master Server gồm 65 kịch bản kiểm thử, chạy tự động trong đường ống CI mỗi lần cập nhật mã nguồn cùng với việc đóng gói Docker cho master và game server.

4.10. Bảng ánh xạ hình vẽ với chức năng cá nhân thực hiện

Bảng tổng hợp đối chiếu toàn bộ các sơ đồ kiến trúc, máy trạng thái, giao diện và luồng xử lý do cá nhân thực hiện tương ứng với các chức năng cụ thể trong phân hệ Master Server TCP và hạ tầng:

Hình vẽ	Tên hình vẽ / biểu đồ	Chức năng cá nhân thực hiện
Hình 3.1	Kiến trúc phân hệ Master Server	Thiết kế kiến trúc tổng thể Master Server TCP thuần không phụ thuộc framework; mô hình luồng đơn (single-threaded logic) kết hợp tiếp nhận I/O bất đồng bộ.
Hình 3.2	Cấu trúc khung bản tin TCP	Thiết kế định dạng đóng khung bản tin tiền tố độ dài (6 byte header: length u32 + msgType u16 + thân JSON) giải quyết triệt để vấn đề dính/tách gói TCP.
Hình 3.3	Bộ đọc khung tin nhị phân	Hiện thực bộ đệm tích lũy (accumulating buffer) và máy trạng thái tách khung; kiểm soát an toàn kích thước tối đa 64 KB và cắt dữ liệu đã xử lý.
Hình 3.4	Trình tự xử lý luồng nghiệp vụ	Thiết kế quy trình định tuyến bản tin qua hàng đợi luồng an toàn, bảo đảm mọi xử lý nghiệp vụ diễn ra tuần tự trên luồng logic duy nhất.
Hình 3.5	Máy trạng thái kết nối TCP	Xây dựng máy trạng thái kết nối client: kiểm soát trạng thái chưa xác thực (30 s), đã xác thực (nhịp tim 15 s, im lặng 45 s) và đóng/thu hồi socket an toàn.
Hình 3.6	Máy trạng thái phòng chơi	Quản lý vòng đời phòng sảnh: khởi tạo, chờ sẵn sàng, phân bổ game server khả dụng, chuyển pha thi đấu và tự động dọn dẹp phòng rỗng.
Hình 3.7	Lược đồ CSDL SQLite	Thiết kế cơ sở dữ liệu lưu trữ tài khoản người chơi (băm mật khẩu bảo mật) và kết quả thi đấu; giữ phiên và phòng trên RAM để tối ưu hiệu năng.
Hình 3.8	Giao diện lập trình API	Thiết kế giao diện lập trình cho client Unity và liên kết máy chủ game; đồng bộ hoá an toàn mọi sự kiện mạng về luồng chính giao diện.
Hình 3.9	Giao diện giám sát và chỉ số tải	Hiện thực cổng TCP giám sát chỉ số thời gian thực và thực hiện kiểm thử tải áp lực: duy trì 1 000 kết nối đồng thời với 81 MB RAM và trễ p99 dưới 8,5 ms.
Hình 3.10	Đường ống CI/CD và hạ tầng	Xây dựng quy trình tự động hoá kiểm thử và đóng gói Docker trên GitHub Actions; triển khai máy ảo production với tường lửa và dịch vụ nền systemd.
Hình 4.1	Sơ đồ tuần tự tiếp nhận kết nối	Hiện thực luồng lắng nghe socket, giới hạn 5 kết nối/IP, hạn tuyệt đối 30 s chưa xác thực và cơ chế trích xuất khung tin liên tục từ luồng byte.
Hình 4.2	Sơ đồ tuần tự đăng ký, đăng nhập	Hiện thực bảo mật 2 lớp mật khẩu (SHA-256 client, bcrypt cost 11 server), giới hạn tần suất 5 lần/phút/IP, khoá tài khoản 15 phút sau 10 lần sai và tạo token an toàn 32 byte.
Hình 4.3	Sơ đồ tuần tự sảnh và cấp vé	Hiện thực cơ chế đẩy trạng thái phòng (push), lựa chọn game server tối ưu theo tải CPU và ký vé vào trận 64 byte HMAC-SHA256 (hạn 60 s).
Hình 4.4	Sơ đồ tuần tự ghép trận	Hiện thực thuật toán tìm phòng sẵn có hoặc xếp hàng đợi định kỳ; cơ chế nói lòng gộp người chờ lâu vào nhóm đông nhất tránh phân mảnh hàng đợi.
Hình 4.5	Sơ đồ tuần tự chat sảnh chờ	Hiện thực tính năng chat chuyển tiếp: giới hạn tần suất 5 tin/10 s, làm sạch ký tự điều khiển/Unicode chống tấn công giao diện và spam.
Hình 4.6	Sơ đồ tuần tự vòng đời Game Server	Hiện thực cơ chế đăng ký bằng khoá bí mật (thời gian hằng), nhận báo sống định kỳ (15 s), tự động thu hồi phòng sau 30 s mất liên lạc và lưu kết quả trận.
Hình 4.7	Sơ đồ tuần tự bảo mật TLS	Hiện thực kênh bảo mật TLS tùy chọn với cơ chế ghim vân tay chứng chỉ SHA-256 (certificate pinning) chống tấn công giả mạo máy chủ.